

Team Construction — Spending Pattern Walkthrough

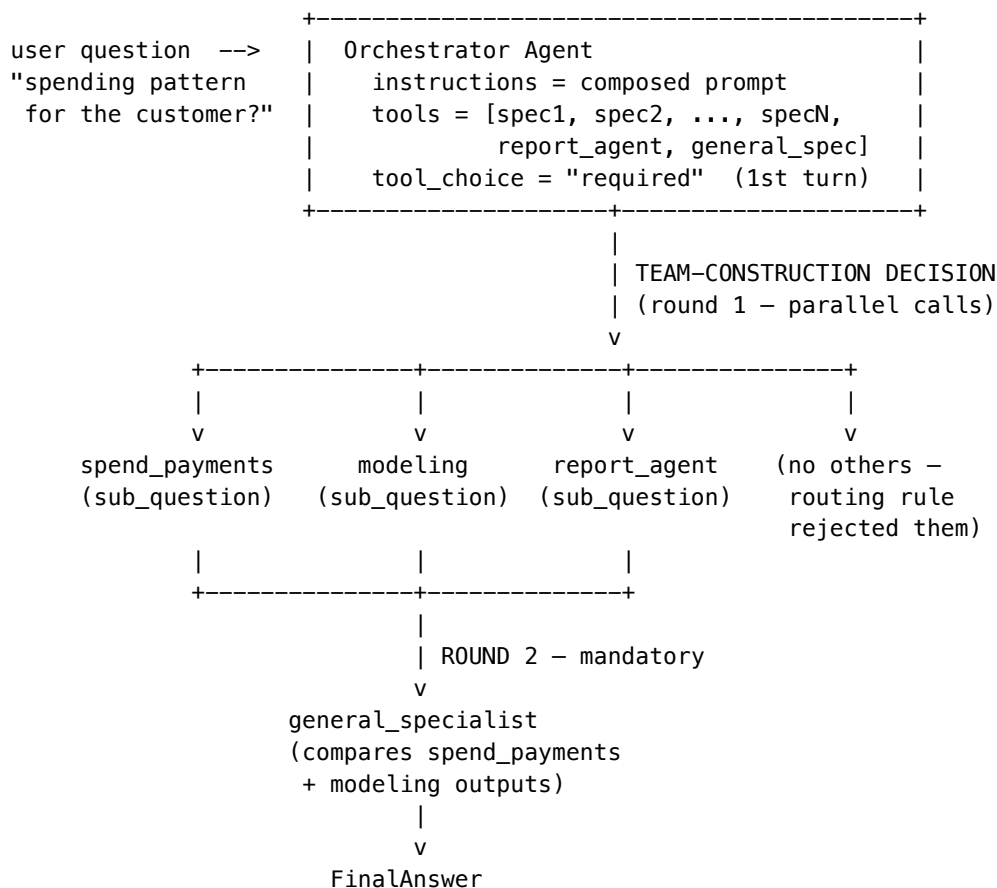
2026-05-06

Team Construction — Spending Pattern Walkthrough

This note walks the team-construction decision: how the orchestrator picks which specialists to call, frames each one's sub-question, and enforces the multi-round protocol that ends in a `FinalAnswer`. It uses the spending-pattern question as the running example. Every callout is anchored to a file and line so the wire is reproducible.

Team construction is not a separate stage with its own JSON output. It IS the orchestrator's first turn — the decision is *expressed* by which tool calls the orchestrator emits, in what shape, and in parallel. The skills, prompt blocks, and `ModelSettings` together steer that one act.

1. Where team construction happens



Two passes through the orchestrator LLM:

1. Round 1: emit a parallel tool-call batch — *N domain specialists
 - `report_agent*`. The team is built here.
2. Round 2: emit `general_specialist(...)` (mandatory whenever round 1 used 2+ domain specialists).

Only after round 2 is the orchestrator allowed to emit a `FinalAnswer`. The protocol is enforced in `agent_factories/orchestrator_agent.py:73–108`.

2. What the orchestrator *sees* at decision time

`build_orchestrator_agent(...)` in `agent_factories/orchestrator_agent.py:145–186` composes the instructions string from five concatenated blocks, plus an auto-generated roster, plus a hard protocol block. Together these are the only inputs to the team-construction decision (the user question is the runtime input).

2.1 The composed system prompt

`_compose_orchestrator_instructions` (lines 62-117) joins, in order:

1. **`skills/workflow/team_construction.md`** — the routing rules (concept -> specialist table, cross-domain topics table, subject-vs-object rule, sub-question framing, follow-up reuse).
2. **`skills/workflow/data_catalog.md`** — what tables exist and what they carry, so routing-by-data-table is grounded.
3. **`skills/workflow/synthesis.md`** — the round-2/round-3 merge contract (referenced now so the orchestrator already knows the downstream synthesis depends on diverse-team coverage).
4. **`skills/workflow/balancing.md`** — how to reconcile `report_agent` vs. `team` vs. `general_specialist`; informs round-1 coverage.
5. The **TOOL-USE DISCIPLINE** block (lines 73-108, hand-authored inline) — the hard protocol gate (see § 2.3).

Then optionally:

6. Pillar **`concept_glossary`** — the same pillar-level vocabulary the specialists see, so orchestrator routing decisions speak the same canonical names as specialist filter construction.
7. Auto-generated **TEAM ROSTER** — § 2.2.

2.2 The auto-generated team roster

`_render_team_roster(specialists, catalog)` at lines 16-59 builds a dynamic, data-grounded routing reference by walking the live agent list. For each specialist agent it:

- loads the matching `DomainSkill` via `load_domain_skill(s.name)`,
- pulls the skill's one-line description,
- iterates `skill.data_hints` (the table names this specialist owns) and looks each up in the live `DataCatalog` for its description,
- surfaces the top 3 `risk_signals` so concept-routing has a fallback.

For the credit pillar this expands to (excerpted):

=== TEAM ROSTER (auto-generated from skills + catalog) ===

- `spend_payments` – Spend & Payments – payment trends, delinquency, spend spikes
 owns ``txn_monthly``: monthly aggregates of spend volume.
 owns ``spends``: transaction-level spend with merchant + industry.
 owns ``payments``: per-payment-attempt with status + return_reason.
 flags risks like: payment < minimum due for 2+ months;
 spend spike > 3x average; ...

- modeling – Modeling domain skill – internal ML risk scores (CDSS, TSR, etc.) and their drivers
owns `model\_scores`: ML model outputs for this case.
owns `score\_drivers`: per-month top/bottom feature attributions.
flags risks like: score drop > 50 points in 3 months; ...
- crossbu – ...
- bureau – ...
- capacity_afford – ...
- customer_rel – ...
- wcc – ...

ROUTING RULE: pick the specialist whose `owns` table most directly carries the reviewer's question. Prefer 1–2 specialists; only widen to 3+ when the question explicitly spans multiple domains.

Without this enrichment the orchestrator would see every specialist tool with the same boilerplate "Domain specialist 'X' – call with a focused sub-question." and route blind. The roster is what lets the LLM match the reviewer's vocabulary ("merchant concentration", "score evolution") to the specialist whose data actually carries the answer.

A second enrichment hits each specialist's *tool-description* string via `_describe_specialist(...)` (lines 120–142). That string is what the SDK shows the orchestrator next to each tool name in the function schema:

Domain specialist 'spend_payments'. Spend & Payments – payment trends, delinquency, spend spikes. Owns data tables: `txn_monthly`, `spends`, `payments`. Call with a focused sub-question scoped to this domain.

So the orchestrator picks tools using both the prompt-level roster and the SDK-level tool descriptions — the same data, two channels.

2.3 The TOOL-USE DISCIPLINE block (hard gates)

This is the part that *makes* the team-construction decision binding. It enforces three rules (`orchestrator_agent.py`: 73–108):

1. Grounding gate — every `FinalAnswer` MUST be preceded by
(a) at least one `report_agent` call AND (b) at least one domain specialist call. No loopholes.
2. Parallel execution — `report_agent` and every selected domain specialist must be emitted in a SINGLE response so they run in parallel.
3. General-specialist gate — when 2+ domain specialists are on the team, `general_specialist` MUST be called in round 2 *after* their results land. Compliance is self-checked: count unique domain specialists emitted this turn; if ≥ 2 , scan for a `general_specialist` tool result; if absent, call it before `FinalAnswer`.

These three rules collapse a free-form decision into a deterministic shape: parallel batch, then optional comparator, then answer.

2.4 What's in the **tools** array

`build_orchestrator_agent` (lines 153–167) wraps every specialist plus `report_agent` plus `general_specialist` via `redacting_tool(...)`:

```
tools = [redacting_tool(s, name=s.name, description=_describe_specialist(s))
         for s in specialists]
tools.append(redacting_tool(report_agent, name="report_agent",
                           description="Look up prior curated reports for this case."))
```

```
tools.append(redacting_tool(general_specialist, name="general_specialist",
                             description="Compare specialist outputs and surface contradictions."))
```

The output type is forced to FinalAnswer:

```
output_type=AgentOutputSchema(FinalAnswer, strict_json_schema=False)
model_settings=ModelSettings(tool_choice="required")
```

tool_choice="required" makes the very first orchestrator turn a tool call (the model can't shortcut a FinalAnswer from prompt text alone). The SDK default reset_tool_choice=True flips this back to "auto" after the first call so synthesis is unblocked.

3. The routing decision for “spending pattern”

When the user question is “*What is the spending pattern for this customer?*”, the orchestrator LLM evaluates the prompt above.

3.1 First match — the cross-domain table

skills/workflow/team_construction.md:31 is the deciding row:

spending / spend pattern / spend behavior / spend trajectory / spend volume / merchant concentration

Specialists to consider: MUST include BOTH **spend_payments** AND **modeling** (+ crossbu only when the question is explicitly B2B)

- **spend_payments**: transaction-level spend AND merchant-name / merchant-industry concentration of the customer's own spending.
- **modeling**: ML-derived spend features (out_of_pattern_spend*, cust_enhnc_one_way_spend_concentration_30 time-weighted spend variables) — pattern-level signals the raw transaction view can't surface alone.
- **crossbu** belongs ONLY when the reviewer asks about the merchant side of the customer's businesses (B2B charge volume), a different concept than the customer's own purchasing behavior.

Three follow-on consequences flow from this row:

- Team size = 2 specialists, not 1. The cross-domain table has authority over the default “1-2 is normal” rule (selection rule #1, team_construction.md:63). A spending-pattern answer with only spend_payments is explicitly called out as incomplete.
- **crossbu** is excluded. The question doesn't mention B2B / charge volume; the routing rule rejects it. The “balance vs spend” edge case at team_construction.md:39 reinforces this — balance lives on crossbu_cards, but the question is about *spend* (a flow), so crossbu is off-team.
- Other specialists are dropped. Selection rule #2 forbids “for context” picks; bureau, wcc, capacity_afford, customer_rel carry no spending data, so they don't justify a tool call.

3.2 Pair with **report_agent** (always)

Selection rule #5 (team_construction.md:67) and the TOOL-USE DISCIPLINE block both require report_agent to fire on the same turn. This is independent of which domain specialists the routing picked — curated prior-report context is grounding for *every* answer.

3.3 The emitted round-1 batch

In one parallel response, the orchestrator emits:

```

spend_payments(sub_question="...")
modeling(sub_question="...")
report_agent(sub_question="...")

```

All three execute concurrently. Each one passes through the `redacting_tool` wrapper (`agent_factories/redacting_tool.py:119`), which sanitizes the sub-question on input, runs the inner agent with `max_turns=25`, then redacts the output.

3.4 Sub-question framing per specialist

`team_construction.md:69–76` lays down five framing rules. Applied to the spending-pattern root question, each specialist gets a sub-question scoped to its data, orthogonal to the others, and using its own vocabulary:

Specialist	Framing principle	Example sub-question
<code>spend_payments</code>	Use <code>spends_data</code> and <code>payments</code> vocabulary; ask the broad pattern question because the domain skill's playbook (<code>spend_payments.md:34–104</code>) already covers temporal shape + merchant concentration + spend-vs-payment ratio.	<i>“Walk through the customer’s spending pattern: monthly volume + count, top recurring and high-value merchants by name, industry mix, per-merchant trends, and spend-vs-successful-payments ratio. Use <code>spends_data.Amount / Date / Merchant Name / Merchant Industry</code> and <code>payments.Payment Amount / payment_status</code>.”</i>
<code>modeling</code>	Use <code>model_scores</code> and <code>score_drivers</code> vocabulary; ask the question the raw spend view can’t answer — what the ML scores see.	<i>“What do the internal model scores say about this customer’s spending pattern? Cover (a) any spend-related drivers in <code>top_<score>*</code> or <code>bottom_<score>*</code> (out-of-pattern spend, spend concentration, time-weighted spend variables), (b) score evolution over the spend window, (c) divergence between model and bureau if relevant.”</i>
<code>report_agent</code>	Curated-report lookup; verbatim or near-verbatim of the root.	<i>“Pull any prior curated narrative about the customer’s spending pattern, recurring merchants, or unusual spend behavior.”</i>

The framings are deliberately non-overlapping: `spend_payments` works the transaction tape; `modeling` works the ML signal; `report_agent` works the prior-narrative file. Selection rule #4 (“orthogonal across specialists — no duplicates”) in `team_construction.md:74` is the formal source of this discipline.

4. Round 2 — **general_specialist**

After all three round-1 results land in the orchestrator’s context, the hard gate in `orchestrator_agent.py:84–107` fires. Because the team had 2+ domain specialists (`spend_payments` and `modeling`), the orchestrator **MUST** emit:

```

general_specialist(sub_question="<the spend_payments findings>;
                        <the modeling findings>;

```

```
compare and surface contradictions or
cross-domain insights")
```

The general specialist (`agent_factories/general_specialist.py`) is tool-less (`tools=[]`) — it does pure synthesis on the in-context specialist outputs and emits a `ReviewReport`. Its skill is `skills/workflow/comparison.md`. The orchestrator is required to read this round-2 result before composing its `FinalAnswer`.

The skill's stated job (`synthesis.md:14`) is narrow: compare team outputs only — *not* compare team-vs-report. That's the orchestrator's job at synthesis time. So the round-2 specialist focuses on questions like:

- Does modeling's out-of-pattern spend signal align with the late-window spend spike `spend_payments` flagged?
 - If `spend_payments` says concentration is rising while modeling says concentration is flat, that's a contradiction the orchestrator's `flags` field must carry forward.
-

5. Compliance self-check before **FinalAnswer**

The TOOL-USE DISCIPLINE block ends with an explicit compliance routine (`orchestrator_agent.py:99–107`):

```
count unique domain specialists called this turn -> N
if N >= 2:
    scan conversation for a general_specialist tool result
    if absent: call general_specialist FIRST, then FinalAnswer
    if present: proceed to FinalAnswer
if N == 1:
    skip general_specialist (nothing to compare)
    proceed to FinalAnswer
```

For a spending-pattern question, `N == 2`, so `general_specialist` is mandatory. Emitting `FinalAnswer` without it is “a protocol violation, will be flagged downstream, and may be rejected” — the prompt's own language.

Synthesis time then folds three inputs into the final answer per `skills/workflow/synthesis.md:10–21`:

1. `report_agent` (curated narrative),
2. `spend_payments` + modeling (live data findings),
3. `general_specialist` (cross-domain reconciliation).

The orchestrator's last decision — what `FinalAnswer.flags` to carry forward — is governed by `balancing.md`.

6. Follow-up turn — team REUSE

If the next reviewer question is a near-paraphrase or same-domain follow-up (“*and how did Industrial Supplies trend month-by-month?*”), selection rule #4 in `team_construction.md:66` applies:

Follow-ups & near-duplicates — REUSE THE PRIOR TEAM. When the new question is in the same domain as the previous turn ... reuse the EXACT SAME team — same specialists, same roles. They retain per-specialist conversation memory across turns within this session (the wrapper preserves each one's input/output history per `AppContext._specialist_histories`), so reusing them lets each specialist build on what they already discovered instead of restarting from cold.

So the round-1 batch becomes:

```
spend_payments(sub_question="now narrow to Industrial Supplies – ...")
modeling(sub_question="and the matching score-driver lens for ...")
report_agent(sub_question="any prior curated note on Industrial Supplies?")
```

`spend_payments` and `modeling` continue their own threads — the `redacting_tool` wrapper prepends their prior turn’s input/output to the new sub-question via `histories[name] = result.to_input_list()` (`agent_factories/redacting_tool.py:110–112`). This is also why the team must be the same: a fresh specialist would start cold, lose the discovery work, and probably re-call `list_available_tables / get_table_schema` from scratch.

The `AppContext._specialist_call_cache` (lines 61-78) further dedupes identical normalized sub-questions within the same context — important when `safechain` mode emits trivial wording variants.

7. What can go wrong (and how the rules catch it)

Failure	Caught by
Routing only <code>crossbu</code> for a “spending pattern” question (a known mis-route — <code>crossbu</code> owns <code>crossbu_cards.balance</code> , a stock quantity, not the <code>spend</code> flow).	<code>team_construction.md:39–41</code> edge-case block + the cross-domain table forcing <code>spend_payments</code> for <code>spend</code> questions.
Answering with <code>spend_payments</code> alone (omitting <code>modeling</code>).	The cross-domain row at line 31 explicitly calls this incomplete; selection rule #1 (line 63) tells the LLM to read the cross-domain table BEFORE applying the “1-2 is normal” default.
Emitting <code>FinalAnswer</code> after only one specialist call (no <code>report_agent</code>).	TOOL-USE DISCIPLINE rule #1 — both <code>report</code> + <code>1+</code> specialist required for grounding.
Skipping <code>general_specialist</code> after 2 domain specialists.	Compliance self-check at <code>orchestrator_agent.py:99–107</code> ; downstream flagging if violated.
Adding bureau “for context”.	Selection rule #2 — every pick must carry weight.
Reshuffling the team for a follow-up.	Selection rule #4 — reuse prior team for near-duplicates so per-specialist memory is preserved.
Hallucinating a <code>FinalAnswer</code> from prompt text alone (no tool calls).	<code>tool_choice="required"</code> on the orchestrator’s first turn forces a tool call.

8. TL;DR

For a spending-pattern question, team construction is a single deterministic decision driven by:

- the cross-domain row in `team_construction.md` (-> `spend_payments` – `modeling`),
- the universal `report_agent` pairing rule,
- the parallel-execution + general-specialist hard gates in the orchestrator’s TOOL-USE DISCIPLINE block,
- and `tool_choice="required"` to force the first turn into a tool call.

Round 1: parallel `spend_payments` + `modeling` + `report_agent` with orthogonal, vocabulary-scoped sub-questions. Round 2: mandatory `general_specialist` to compare. Then `FinalAnswer`. On follow-ups in the same domain, the EXACT SAME team is reused so each specialist continues its own thread via `AppContext._specialist_histories`.

The skill, prompt, and `ModelSettings` together collapse a free-form team-selection problem into a small number of allowed shapes — the orchestrator’s “creativity” is reduced to picking which specialists match the question, leaving structure and ordering to the rules.